



ČLAN 1 POROČILO

Iris Skledar Hudoklin



26 MAY 2026

OSNOVE RAČUNALNIŠKEGA VIDA

Program RIT-VS, 2. letnik

Člani skupine MIN Deliveries : Manja Zdouc, Iris Skledar Hudoklin, Nika Zajšek

I. Kazalo vsebine

II.	Kazalo slik	0
1	Uvod	1
2	Zajem podatkov	2
2.1	Android vmesnik.....	2
2.2	Python skripta (zajem_podatkov.py)	3
2.2.1	Haar Cascade	5
3	Osnovna predobdelava (predobdelava.py).....	5
3.1	Integracijski cevovod za API (pripravi_sliko_za_api)	6
3.1.1	Lokalizacija in inteligentno izrezovanje (ROI)	7
3.1.2	Standardizacija geometrije in čiščenje podatkov	7
3.1.3	Normalizacija in priprava tenzorja	8
4	Augmentacija podatkov (Točka 5 navodil)	9
4.1	Funkcije za augmentacijo podatkov.....	10
4.1.1	odstrani_sum()	10
4.1.2	dodaj_sum()	10
4.1.3	spremeni_svetlost()	11
4.1.4	rotiraj_sliko()	11
4.2	predobdelaj_podatke().....	13
4.2.1	Validacija	13
4.2.2	Glavni cevovod za obdelavo in sledenje podatkom.....	14
4.2.3	Izvajanje sistema za augmentacijo in generiranje končnega nabora	16
5	Priprava učnih množic (Točka 6 navodil)	17
5.1	Tehnična izvedba razdelitve podatkov	18
6	Rezultati in dokazi	19
7	Zaključek.....	23
8	Viri	24

II. Kazalo slik

Slika 1 Android vmesnik postavitvev Faceld	2
Slika 2 Faceld zaslon: Android vmesnik demonstracija	2
Slika 3 Detekcija obraza (zajem_podatkov.py)	3
Slika 4 Zrcaljene in pretvorba v sivinsko sliko (zajem_podatkov.py)	3
Slika 5 iskanje obrazov, padding (zajem_podatkov.py)	4
Slika 6 funkcija pripravi_sliko_za_api	6
Slika 7 izrezovanje v pripravi_sliko_za_api()	7
Slika 8 čiščenje slike v pripravi_sliko_za_api()	7
Slika 9 odstranjena normalizacija v pripravi_sliko_za_api()	8
Slika 10 rezultat funkcije pripravi_sliko_za_api()	8
Slika 11 rotirana slika	9
Slika 12 original slika	9
Slika 13 zrnata/šumna slika	9
Slika 14 zamegljena slika	9
Slika 15 posvetljena slika	9
Slika 16 rotacija, zrnatost in potemnitev	9
Slika 17 rotacija in zrnatost	9
Slika 18 rotacija in močna posvetlitev	9
Slika 19 rotacija in potemnitev	9
Slika 20 funkcija odstrani_sum()	10
Slika 21 funkcija dodaj_sum()	10
Slika 22 funkcija spremeni_svetlost()	11
Slika 23 funkcija rotiraj_sliko()	11
Slika 24 validacija v funkciji predobdelaj_podatke()	13
Slika 25 obdelava in sledenje podatkov v predobdelaj_podatke()	14
Slika 26 generiranje končnega nabora v predobdelaj_podatke()	16
Slika 27 razdelitev podatkov v predobdelaj_podatke()	18
Slika 28 commit testiranja zajem_podatkov.py	19
Slika 31 surova slika	19
Slika 32 obdelana slika	19
Slika 29 terminal ob zagonu predobdelava.py	20
Slika 30 struktura map	20
Slika 33 FaceldActivity	21
Slika 34 README.md navodila	22

1 Uvod

V okviru projektne naloge Smart Flower Locker (pametni paketnik) pri predmetu Osnove računalniškega vida, sem prevzela vlogo Člana 1, ki je odgovoren za zajem, pripravo in augmentacijo podatkov ter del aplikacijske logike.

Moje specifične zadolžitve, ki sem jih v celoti izpolnila, so obsegale:

- **Pripravo postopka in razvoj programske opreme za zajem:** Razvila sem Python skripto `zajem_podatkov.py` za namizni zajem ter namensko Android aktivnost `FacelIdActivity` za mobilni zajem učnih slik.
- **Organizacijo podatkov v smiselno strukturo:** Vzpostavila sem hierarhični sistem map (`surovi_podatki`, `obdelani_podatki`, `final_split`), ki omogoča pregledno sledenje podatkom različnih uporabnikov.
- **Osnovno predobdelavo slik:** Implementirala sem algoritme za glajenje digitalnega šuma (denoising), standardizacijo velikosti slik na 224x224 pikslov ter pretvorbo v različne barvne prostore (RGB in Grayscale).
- **Implementacijo lastnih postopkov augmentacije:** Zasnovala in kodirala sem napredne filtre za umetno povečanje nabora podatkov, vključno z rotacijami, spreminjanjem ekspozicije, dodajanjem sintetičnega šuma (grainy efekt) in zrcaljenjem.
- **Pripravo učne, validacijske in testne množice:** Avtomatizirala sem proces naključnega razvrščanja podatkov v razmerju 70/15/15, kar je ključno za objektivno vrednotenje modela.
- **Integracijo v končno aplikacijo:** V Android aplikacijo sem dodala logiko za upravljanje z varnostnimi dovoljenji (kamera, obvestila) ter povezala biometrični zajem z uporabniškim vmesnikom za prijavo in registracijo.

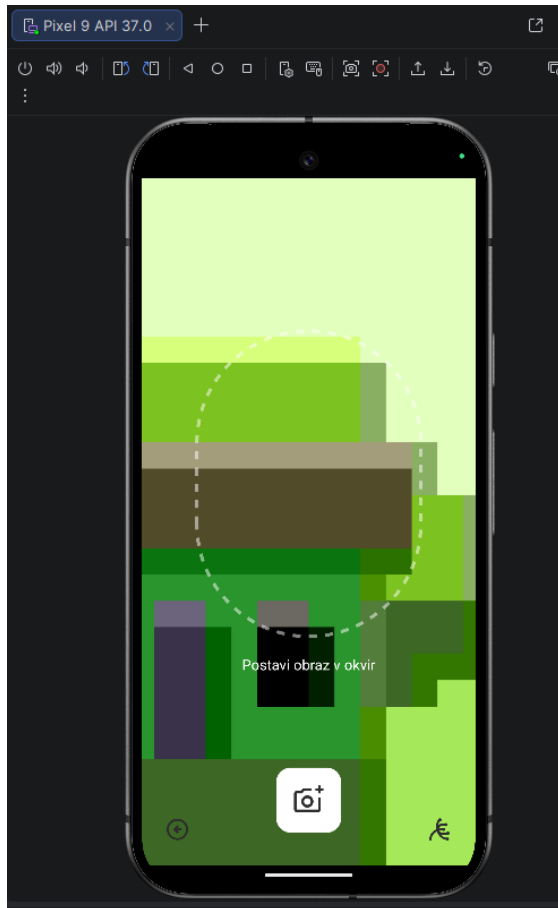
Vsa orodja sem razvila v jeziku Python z uporabo knjižnice OpenCV, mobilni del pa v jeziku Kotlin znotraj okolja Android Studio. V nadaljevanju poročila so podrobneje predstavljeni tehnični vidiki izvedbe teh nalog.

2 Zajem podatkov

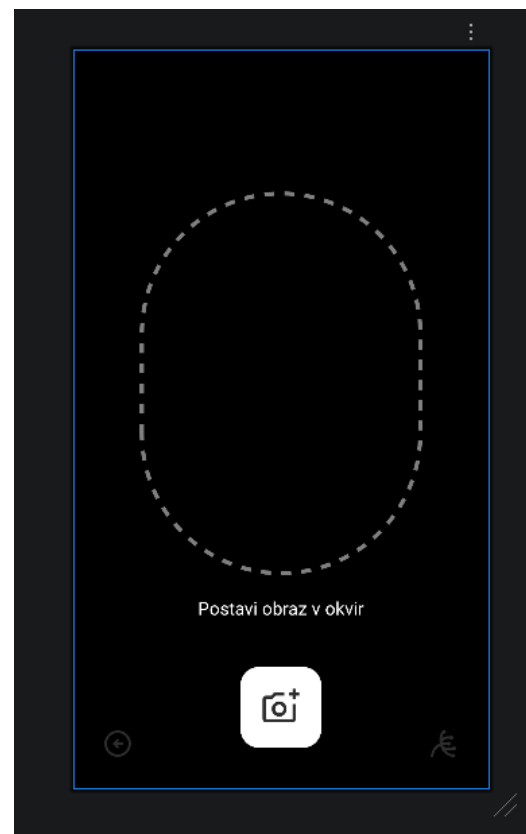
Zajem sem izvedla na dva načina:

2.1 Android vmesnik

Razvila sem namenski zaslon (FacelIdActivity), ki uporablja kamero mobilne naprave. Ključni element je vizualno vodilo (ovalni overlay), ki uporabnika prisili, da obraz postavi v sredino, kar zmanjša geometrijska odstopanja že ob samem zajemu.



Slika 2 FacelId zaslon: Android vmesnik demonstracija



Slika 1 Android vmesnik postavitvev FacelId

2.2 Python skripta (zajem_podatkov.py)

Za hitro gradnjo začetnega nabora podatkov sem napisala skripto v Pythonu, ki uporablja knjižnico OpenCV. Skripta uporablja Haar Cascade klasifikator za detekcijo obraza v realnem času.

```
# Uporabi vgrajen Haar Cascade za detekcijo obraza (standard)
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
```

Slika 3 Detekcija obraza (zajem_podatkov.py)

- **cv2.CascadeClassifier(...)** je funkcija iz knjižnice OpenCV, ki ustvari (inicializira) klasifikator. Kot argument pričakuje pot do datoteke XML, v kateri so shranjena navodila (matematični parametri), kako prepoznati določen objekt.
- **cv2.data.harcascades** je vgrajena pot v knjižnici OpenCV. Ker ima vsak računalnik knjižnice nameščene drugje, ta ukaz samodejno poišče mapo na disku, kjer so shranjeni vsi vnaprej naučeni modeli (npr. za obraz, oči, nasmeh, celo telo). Primer poti, ki jo ta ukaz vrne: C:\Python\Lib\site-packages\cv2\data\
- **+ 'haarcascade_frontalface_default.xml'** pa s simbolom + združimo pot do mape in točno ime datoteke modela. Ta specifična datoteka vsebuje tisoče Haarovih značilnosti (pravokotnih filtrov), ki so se med učenjem izkazale za najboljše pri prepoznavanju človeškega obraza od spredaj.

```
# Zrcaljenje slike (za lažje pozicioniranje)
frame = cv2.flip(frame, 1)
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Slika 4 Zrcaljene in pretvorba v sivinsko sliko
(zajem_podatkov.py)

- **Prva vrstica cv2.flip**, vodoravno zrcali sliko iz kamere, ker standardne kamere vračajo obrnjeno sliko. Z vodoravnim zrcaljenjem (parameter 1) dosežemo učinek ogledala, kar uporabniku omogoča naravno in intuitivno pozicioniranje obraza v okvir.
- **Druga vrstica cvtColor(frame, cv2.COLOR_BGR2GRAY)** pretvori barvno sliko (v BGR formatu) v sivinsko (črno-belo), ker večina detekcijskih algoritmov (vključno s Haar Cascade) ne potrebuje barvnih informacij za prepoznavo struktur (oči, nos, usta). Sivinska slika vsebuje 3-krat manj podatkov, kar omogoča hitrejšo obdelavo in večjo odzivnost v realnem času.

```
# Iskanje obrazov
faces = face_cascade.detectMultiScale(gray, 1.3, 5)

trenutni_obraz = []

for (x, y, w, h) in faces:
    # Doda 30% zamika (padding)
    offset_w = int(w * 0.3)
    offset_h = int(h * 0.3)
    # Izračuna nove koordinate
    y1 = max(0, y - offset_h)
    y2 = min(frame.shape[0], y + h + offset_h)
    x1 = max(0, x - offset_w)
    x2 = min(frame.shape[1], x + w + offset_w)

    trenutni_obraz.append((x1, y1, x2, y2))

    # Nariše moder kvadrat okoli obraza (guideline)
    cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 0, 0), 2)
```

Slika 5 iskanje obrazov, padding (zajem_podatkov.py)

- Prva vrstica na sliki **face_cascade.detectMultiScale(gray, 1.3, 5)** je dejanski ukaz za iskanje obrazov
 - **1.3 (Scale Factor)** določa, za koliko se slika pomanjša ob vsakem koraku iskanja (omogoča zaznavo obrazov različnih velikosti).
 - **5 (Min Neighbors)** pove, kolikokrat mora algoritem potrditi obraz na istem mestu, da ga sprejme kot pravega (preprečuje lažne zadetke).
- Vrstico **offset_w = int(w * 0.3)** in **offset_h = int(h * 0.3)** sem dodala, ker Haar Cascade običajno vrne zelo tesen pravokotnik (le oči, nos in usta). Kot Član 1 sem dodala 30 % zamika, da zajamem širšo strukturo glave (čelo, brado, ušesa), kar je nujno za kakovostno učenje modela.
- Naslednje vrstice (y1,y2,x1,x2) ki vsebujejo **max(0, ...)** in **min(frame.shape, ...)** preprečujejo sesutje programa. Če se uporabnik preveč približa robu kamere, bi izračunane koordinate z zamikom lahko padle izven slike (npr. negativne vrednosti). Ti ukazi poskrbijo, da se koordinata vedno ustavi na robu slike (0 ali max širina/višina).
- In zadnja vrstica na sliki **cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 0, 0), 2)** na zaslon nariše moder pravokotnik. To služi kot uporabniško vodilo (User Experience), da oseba ve, kdaj jo je sistem prepoznal in ali stoji pravilno.

2.2.1 Haar Cascade

Haar Cascade je algoritem za detekcijo objektov, ki temelji na strojnem učenju. Deluje na principu iskanja specifičnih vzorcev (t.i. Haarovih značilnosti) na sliki. Te značilnosti prepoznajo razlike v intenziteti svetlobe med sosednjimi pravokotnimi območji – na primer, območje oči je običajno temnejše od območja lic, nosni koren pa svetlejši od oči.

Beseda 'Cascade' (kaskada) pomeni, da je klasifikator sestavljen iz več stopenj. Slika potuje skozi serijo filtrov: če prva stopnja ugotovi, da na določenem delu slike verjetno ni obraza, ta del takoj zavrže. To omogoča izjemno hitro obdelavo v realnem času, saj se procesorska moč porablja le za tiste dele slike, ki dejansko spominjajo na obraz.

Za uporabo Haar Cascade klasifikatorja sem se odločila iz naslednjih razlogov:

- **Hitrost in učinkovitost:** Algoritem je izjemno hiter in omogoča detekcijo v realnem času brez potrebe po močni grafični kartici (GPU), kar je idealno za skripto, ki teče na prenosniku med zajemom slik.
- **Robustnost za osnovno detekcijo:** Ker je moja naloga (Član 1) zajem surovih podatkov in ne končna razpoznavna identitete, je Haar Cascade več kot dovolj natančen za iskanje koordinat obraza in samodejno izrezovanje slik.
- **Dostopnost:** Je standardni del knjižnice OpenCV z dobro dokumentirano podporo in vnaprej naučenimi modeli, kar zagotavlja stabilnost procesa zajema.

3 Osnovna predobdelava (predobdelava.py)

Surove slike so bile podvržene naslednjim postopkom (Točka 4 navodil):

- **Sprememba velikosti (Resizing):** Vse slike so bile skalirane na **224x224 pikslov**, kar je standardna vhodna dimenzija za večino sodobnih arhitektur (npr. MobileNet ali ResNet).
- **Odstranjevanje šuma (Denoising):** Uporabila sem **Gaussov filter** (cv2.GaussianBlur), da sem zgladila digitalni šum, ki nastane pri zajemu v slabših svetlobnih pogojih.

- **Pretvorba barvnih prostorov:** Poleg RGB slik sem generirala tudi sivinske (**Grayscale**) različice, kar omogoča modelu učenje na strukturnih značilnostih brez barvne pristranskosti.
- **Normalizacija:** Vrednosti slikovnih pik so bile pripravljene za pretvorbo iz ranga [0, 255] v [0, 1], kar izboljša stabilnost učenja modela.

3.1 Integracijski cevovod za API (pripravi_sliko_za_api)

Da bi zagotovili popolno usklajenost med fazo učenja in fazo realne uporabe (inferenca), sem razvila funkcijo, ki izvaja identične operacije na surovi sliki, ki jo API prejme od mobilne naprave.

```
# --- NOVA FUNKCIJA ZA ČLANA 3 (API) ---
def pripravi_sliko_za_api(img):
    """
    To funkcijo uporabi Član 3 v API-ju.
    Vzame surovo sliko iz telefona, najde obraz, ga izreže in pripravi za model.
    Normalizacijo na tensor opravi model_loader.predict().
    """

    # naloži detektor (isti kot v zajem_podatkov.py)
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    if len(faces) > 0:
        # Vzame prvi zaznan obraz
        (x, y, w, h) = faces[0]

        # Doda isti padding (30%) da je slika enaka ko pri učenju
        offset = int(w * 0.3)
        y1, y2 = max(0, y-offset), min(img.shape[0], y+h+offset)
        x1, x2 = max(0, x-offset), min(img.shape[1], x+w+offset)

        img_obdelana = img[y1:y2, x1:x2]
        obraz_najden = True
    else:
        # Če obraza ne najde vrne original (ali pa API lahk javi napako)
        img_obdelana = img
        obraz_najden = False

    # OSNOVNA PREDOBDELAVA
    target_size = (224, 224)
    # 1. Resizing po cropu
    img_obdelana = cv2.resize(img_obdelana, target_size)
    # 2. Denoising (isti filter kot pri učenju)
    img_obdelana = odstrani_sum(img_obdelana)

    return img_obdelana, obraz_najden
```

Slika 6 funkcija pripravi_sliko_za_api

3.1.1 Lokalizacija in inteligentno izrezovanje (ROI)

Prvi del funkcije uporabi vnaprej pripravljen detektor za iskanje obraza na surovi sliki (tako kot smo to naredili tudi v `zajem_podatkov.py`). Ključni prispevek v tem delu je **implementacija 30 % zamika (paddinga)** okoli zaznanih koordinat. S tem zagotovimo, da slika, ki gre v model, ni preveč tesna in vsebuje vse strukturne elemente glave, ki so nujni za zanesljivo razpoznavo identitete.

```
# naloži detektor (isti kot v zajem_podatkov.py)
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.3, 5)

if len(faces) > 0:
    # Vzame prvi zaznan obraz
    (x, y, w, h) = faces[0]

    # Doda isti padding (30%) da je slika enaka ko pri učenju
    offset = int(w * 0.3)
    y1, y2 = max(0, y-offset), min(img.shape[0], y+h+offset)
    x1, x2 = max(0, x-offset), min(img.shape[1], x+w+offset)

    img_obdelana = img[y1:y2, x1:x2]
    obraz_najden = True
else:
    # Če obraza ne najde vrne original (ali pa API lahk javi napako)
    img_obdelana = img
    obraz_najden = False
```

Slika 7 izrezovanje v `pripravi_slika_za_api()`

3.1.2 Standardizacija geometrije in čiščenje podatkov

V drugem koraku funkcija izvede spremembo velikosti (Resizing) na fiksno dimenzijo 224x224 pikslov. Takoj za tem se uporabi funkcija `odstrani_sum()`, ki z uporabo Gaussovega filtra zgladi sliko. Ta korak je kritičen, saj mobilne kamere pogosto generirajo visokofrekvenčni digitalni šum, ki bi brez čiščenja negativno vplival na natančnost predvidenega modela.

```
# OSNOVNA PREDOBDELAVA
target_size = (224, 224)
# 1. Resizing po cropu
img_obdelana = cv2.resize(img_obdelana, target_size)
# 2. Denoising (isti filter kot pri učenju)
img_obdelana = odstrani_sum(img_obdelana)
```

Slika 8 čiščenje slike v `pripravi_slika_za_api()`

3.1.3 Normalizacija in priprava tenzorja

Zadnja faza obdelave je matematična normalizacija. Slikovne pike pretvorim v tip float32 in jih skaliram na interval [0, 1]. Rezultat funkcije je t.i. slikovni tenzor, ki je neposredno pripravljen za vhod v nevronske mreže, ne da bi Član 2 ali Član 3 potrebovala dodatno logiko za obdelavo podatkov na strežniku.

```
# 3. Normalizacija na [0,1]
img_tensor = img_obdelana.astype('float32') / 255.0
```

Slika 9 odstranjena normalizacija v pripravi_slika_za_api()

To smo kasneje odstranili, ker se je član 2 v svoji skripti odločil uporabiti normalizacijo in je postala v predobdelava.py odvečna. Na koncu pa je funkcija vrnila še obdelano oz. očiščeno sliko in podatek ali je obraz najden (true/false).

```
return img_obdelana, obraz_najden
```

Slika 10 rezultat funkcije pripravi_slika_za_api()

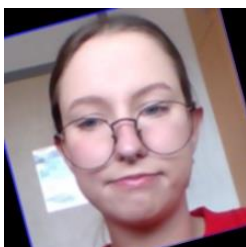
4 Augmentacija podatkov (Točka 5 navodil)

Zaradi omejenega števila fizičnih zajemov sem nabor podatkov sintetično povečala iz 90 surovih slik na več kot **1300 vzorcev**. Implementirala sem v predobdelava.py naslednje transformacije:

- **Geometrijske:** Horizontalno zrcaljenje (flip) in rotacije pod koti od -20° do $+20^\circ$.

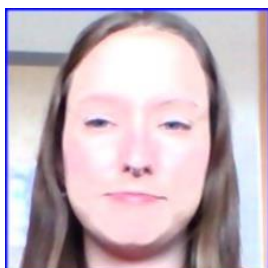


Slika 12 original slika



Slika 11 rotirana slika

- **Fotometrične:** Spreminjanje ekspozicije (exposure up/down) preko HSV barvnega prostora in dodajanje Gaussovega šuma (grainy efekt).



Slika 15 posvetljena slika

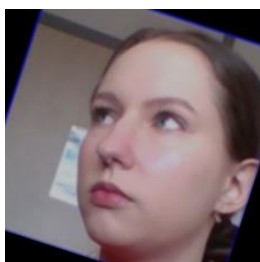


Slika 13 zrnata/šumna slika



Slika 14 zamegljena slika

- **Kombinirane transformacije:** Generiranje slik, ki združujejo več filtrov hkrati, kar simulira najzahtevnejše realne pogoje uporabe.



Slika 19 rotacija in potemnitev



Slika 18 rotacija in močna posvetlitev



Slika 17 rotacija in zrnastost



Slika 16 rotacija, zrnastost in potemnitev

Sistem sem zasnovala tako, da sošolki (Članu 2) ni bilo treba skrbeti za količino podatkov, saj je skripta samodejno pripravila uravnotežen nabor več kot 1300 slik, razdeljenih v standardne množice za učenje.

4.1 Funkcije za augmentacijo podatkov

4.1.1 odstrani_sum()

```
def odstrani_sum(img):  
    """Odstranjevanje šuma (Denoising)."""  
    return cv2.GaussianBlur(img, (3, 3), 0)
```

Slika 20 funkcija odstrani_sum()

Za odstranjevanje digitalnega šuma sem uporabila Gaussov filter z velikostjo jedra 3x3. Ta velikost je optimalna, saj sliko dovolj zgladi za potrebe učenja modela, hkrati pa ohrani ključne strukturne detajle obraza, standardni odklon pa se izračuna samodejno (parameter 0).

4.1.2 dodaj_sum()

```
def dodaj_sum(img):  
    """Simulira grainy (zrnat) efekt."""  
    row, col, ch = img.shape  
    gauss = np.random.normal(0, 15, (row, col, ch))  
    noisy = img + gauss  
    return np.clip(noisy, 0, 255).astype(np.uint8)
```

Slika 21 funkcija dodaj_sum()

Za povečanje robustnosti modela sem implementirala funkcijo za umetno dodajanje šuma. Funkcija generira naključno matriko motenj s standardnim odklonom 15 (intenzivnost šuma) in jo prišteje originalni sliki. Pri tem z uporabo funkcije np.clip zagotovim, da vrednosti slikovnih pik ostanejo znotraj veljavnega razpona [0, 255]. Ta augmentacija simulira realne pogoje slabo osvetljenih prostorov in kamer slabše kakovosti.

4.1.3 spremeni_svetlost()

```
def spremeni_svetlost(img, faktor):  
    """Simulira exposure up/down preko HSV barvnega prostora."""  
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)  
    hsv = np.array(hsv, dtype=np.float64)  
    hsv[:, :, 2] = hsv[:, :, 2] * faktor  
    hsv[:, :, 2][hsv[:, :, 2] > 255] = 255  
    hsv = np.array(hsv, dtype=np.uint8)  
    return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
```

Slika 22 funkcija spremeni_svetlost()

Za simuliranje različnih svetlobnih pogojev (npr. zunanja dnevna svetloba ali slabo osvetljen prostor) sem razvila funkcijo, ki varno manipulira z svetlostjo slike. Namesto neposrednega množenja RGB kanalov, ki bi popačilo kožne tone obraza, funkcija sliko najprej pretvori v HSV barvni prostor.

S tem ločim informacijo o barvi od informacije o svetlobi. Množenje s faktorjem se izvede izključno na tretjem kanalu (Value), ki predstavlja svetlost slike. Da preprečim prekoračitev vrednosti (overflow) nad 255 pri posvetlitvi, vrednosti začasno pretvorim v tip float64, pomnožim s faktorjem in nato vrednosti omejim (clip) na največ 255. Rezultat je naravno posvetljena ali potemnjena slika brez barvnih popačenj.

4.1.4 rotiraj_sliko()

```
def rotiraj_sliko(img, kot):  
    """Rotira sliko za določen kot brez rezanja robov."""  
    (h, w) = img.shape[:2]  
    center = (w // 2, h // 2)  
    M = cv2.getRotationMatrix2D(center, kot, 1.0)  
    return cv2.warpAffine(img, M, (w, h))
```

Slika 23 funkcija rotiraj_sliko()

Uporabniki mobilne aplikacije naprave se redko držijo popolnoma vodoravno ali navpično, zato je za robustnost sistema ključno, da model prepozna obraz tudi ob rahlem nagibu. Implementirala sem funkcijo za geometrijsko rotacijo slike.

Funkcija najprej izračuna točno središče slike ($w // 2$, $h // 2$), kar zagotovi, da se rotacija izvede okoli sredine obraza in ne okoli koordinatnega izhodišča slike. Z uporabo OpenCV funkcije `getRotationMatrix2D` generiram afin matriko za podani kot vrtenja brez spreminjanja povečave ($scale = 1.0$). Transformacijo nato apliciram s funkcijo `warpAffine`, pri čemer izhodna slika obdrži standardne dimenzije (224x224 pikslov). V glavni zanki sem to funkcijo uporabila za generiranje variacij obrazov pod koti -15° , -8° , 8° in 15° .

Za izvedbo rotacije sem namensko izbrala vgrajeni OpenCV funkciji `getRotationMatrix2D` in `warpAffine`.

- Kaj delata: Prva funkcija (**`getRotationMatrix2D`**) je matematično orodje, ki na podlagi naših vhodnih podatkov (središče, kot vrtenja in faktor povečave) izračuna 2×3 afino transformacijsko matriko (To je 2D geometrijska preslikava -v mojem primeru rotacija in premik v središče, ki ohranja vzporednost linij na sliki, zato obraz po rotaciji ostane proporcionalno povsem nespremenjen in se ne popači.). Druga funkcija (**`warpAffine`**) pa to matriko dejansko aplicira in prestavi vsako slikovno piko vhodne slike na njeno novo koordinato.

Zakaj sta izbrani: Ker je njuna glavna prednost izjemna hitrost in optimizacija, saj se v zaledju izvajata v jeziku C/C++. To je bilo pri moji nalogi ključno, saj je morala skripta pri augmentaciji hitro in učinkovito obdelati in transformirati več kot 1300 slik, ne da bi pri tem preobremenila procesor.

4.2 predobdelaj_podatke()

4.2.1 Validacija

```
# 1. PREVERJANJE POGOJEV (Data Validation)
if not os.path.exists(raw_path):
    print("[!] Napaka: Mapa 'surovi_podatki' ne obstaja!")
    return

osebe = [f for f in os.listdir(raw_path) if os.path.isdir(os.path.join(raw_path, f))]

# Preveri če so vsaj 3 osebe (članice ekipe)
if len(osebe) < 3:
    print(f"[!] STOP: Premalo map oseb. Potrebujemo vsaj 3 (najdeno: {len(osebe)}).")
    return
vse_ok = True

# Čiščenje prejšnjih obdelav
if os.path.exists(proc_path): shutil.rmtree(proc_path)
if os.path.exists(split_path): shutil.rmtree(split_path)
os.makedirs(proc_path)

vse_generirane_slike = [] # Seznam za poznejšo razdelitev v množice

for oseba in osebe:
    pot = os.path.join(raw_path, oseba)
    st_slik = len([f for f in os.listdir(pot) if f.lower().endswith(('.jpg', '.jpeg', '.png'))])
    if st_slik < 30:
        print(f"[!] STOP: Mapa '{oseba}' ima premalo slik ({st_slik}/30). Dopolnite nabor.")
        vse_ok = False

if not vse_ok:
    return

print(f"--- Vsi pogoji izpolnjeni. Začenjam generiranje datasea za {len(osebe)} osebe. ---")
```

Slika 24 validacija v funkciji predobdelaj_podatke()

Da bi zagotovila stabilnost sistema in preprečila napake v kasnejši fazi učenja modela, sem na sam začetek skripte za predobdelavo implementirala strog sistem validacije podatkov. Koda deluje po naslednjih korakih:

- Preverjanje obstoja mape in števila oseb: Algoritem najprej preveri, ali mapa surovi_podatki sploh obstaja. Nato z uporabo strnjenege seznama (list comprehension) in knjižnice os prešteje podmape. V skladu z navodili projekta skripta zahteva, da so v mapi zbrani podatki vsaj treh različnih oseb (članov ekipe). Če ta pogoj ni izpolnjen, se izvajanje varno prekine.
- Čiščenje prejšnjih obdelav: Zelo pomemben korak je čiščenje (resetiranje) okolja. Če uporabnik skripto zažene večkrat, bi se nove slike lahko pomešale s starimi. Uporabila sem funkcijo `shutil.rmtree()`, ki v celoti izbriše stari mapi obdelani_podatki in final_split, ter ju nato ustvari povsem na novo. To zagotavlja t. i. idempotentnost – ne glede na to, kolikokrat zaženemo kodo, bo rezultat vedno čist in konsistenten.
- Preverjanje količine slik: Skripta nato z zanko iterira čez vsako zaznano osebo in prešteje datoteke s slikovnimi končnicami (.jpg, .jpeg, .png). Ponovno v skladu z

navodili zahteva minimalno 30 originalnih slik na osebo. V primeru pomanjkanja podatkov skripta zabeleži napako (zastavica vse_ok = False), vendar preveri še ostale osebe, da uporabniku vrne celovito povratno informacijo, kje natančno manjkajo podatki.

Zaradi tega varnostnega mehanizma je preostanek kode (augmentacija in razdelitev) zaščiten in se izvede samo takrat, ko imamo zagotovljene pravilne vhodne podatke.

4.2.2 Glavni cevovod za obdelavo in sledenje podatkom

```
# Gre čez vse mape oseb (iris, nika, ipd.)
for oseba in osebe:
    oseba_raw_dir = os.path.join(raw_path, oseba)
    oseba_proc_dir = os.path.join(proc_path, oseba)
    if not os.path.exists(oseba_proc_dir):
        os.makedirs(oseba_proc_dir)

    print(f"\nObdelujem osebo: {oseba}...")

    for img_name in os.listdir(oseba_raw_dir):
        img_path = os.path.join(oseba_raw_dir, img_name)
        img = cv2.imread(img_path)
        if img is None:
            continue

        # 1. SPREMEMBA VELIKOSTI (Resizing)
        img = cv2.resize(img, target_size)
        # --- ODSTRANJEVANJE ŠUMA ---
        img = odstrani_sum(img)

        ime_osnova = os.path.splitext(img_name)[0]

        # Pomožna funkcija za shranjevanje in sledenje za končni split
        def shrani_in_zabelezi(image, koncnica):
            ime_datoteke = f"{ime_osnova}_{koncnica}.jpg"
            pot_shranjevanja = os.path.join(oseba_proc_dir, ime_datoteke)
            cv2.imwrite(pot_shranjevanja, image)
            vse_generirane_slike.append((pot_shranjevanja, oseba))
```

Slika 25 obdelava in sledenje podatkov v predobdelaj_podatke()

Jedro predobdelave predstavlja glavna zanka, ki avtomatizira branje, osnovno čiščenje in shranjevanje slik. Kodo sem strukturirala tako, da dinamično prebira mape in datoteke, kar sistemu omogoča, da brez spreminjanja kode obdela poljubno število oseb.

Cevovod deluje po naslednjih korakih:

- Iteracija in upravljanje map: Zunanja zanka se sprehodi čez vse zaznane osebe in zanje samodejno ustvari ustrezne podmape v direktoriju obdelani_podatki.
- Branje in zaščita pred napakami: Notranja zanka prebere vsako sliko z uporabo funkcije `cv2.imread()`. Vključila sem varnostni pogoj `if img is None: continue`, ki prepreči sesutje programa, če bi se v mapi slučajno znašla poškodovana ali neveljavna slikovna datoteka.
- Osnovna standardizacija: Takoj po uspešnem branju se slika skalira na ciljno velikost (224x224) in se očisti šuma, da so vse nadaljnje augmentacije izvedene na enotni in kakovostni osnovi.
- Sledenje podatkom (Pomožna funkcija `shrani_in_zabelezi`): Da bi se izognila podvajanju kode pri kasnejšem shranjevanju na desetine različnih augmentacij, sem definirala lokalno pomožno funkcijo. Ta funkcija:
 - Dinamično sestavi novo ime datoteke tako, da originalnemu imenu doda končnico uporabljenega filtra (npr. iz `iris_1.jpg` nastane `iris_1_blur.jpg`).
 - Fizično shrani sliko na disk.
 - Ključen korak za strojno učenje: V globalni seznam `vse_generirane_slike` shrani terko s potjo do nove slike in oznako osebe (`pot_shranjevanja`, `oseba`). S tem sem zagotovila, da sistem točno ve, komu pripada katera slika, kar je kasneje omogočilo pravilno naključno razdelitev v Train, Val in Test množice.

4.2.3 Izvajanje sistema za augmentacijo in generiranje končnega nabora

```
# --- GENERIRANJE VARIACIJ (Augmentacija) -----

# 2. Original in Grayscale (Sivinska) (če rabi Član 2)
shrani_in_zabelezi(img, "orig")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
shrani_in_zabelezi(gray, "gray")

# 3. Zrcaljenje (Flip)
flipped = cv2.flip(img, 1)
shrani_in_zabelezi(flipped, "flip")

# 3. Rotacije (Malo levo, malo desno)
for kot in [-15, -8, 8, 15]:
    rot = rotiraj_sliko(img, kot)
    shrani_in_zabelezi(rot, f"rot{kot}")

# 4. Svetlost (Exposure up/down)
shrani_in_zabelezi(spremeni_svetlost(img, 1.4), "bright")
shrani_in_zabelezi(spremeni_svetlost(img, 0.6), "dark")

# 5. Blur (Zameglitev) in Grainy (Šum)
blur = cv2.GaussianBlur(img, (7, 7), 0)
shrani_in_zabelezi(blur, "blur")
grainy = dodaj_sum(img)
shrani_in_zabelezi(grainy, "grainy")

# 6. KOMBINACIJE (Zmeša filtre)
# Ustvari še 5 unikatnih kombinacij
for i in range(5):
    # Naključna rotacija + naključna svetlost
    c_img = rotiraj_sliko(img, random.randint(-20, 20))
    c_img = spremeni_svetlost(c_img, random.uniform(0.5, 1.5))
    # 50% možnosti, da dodamo še zrnatost
    if random.random() > 0.5:
        c_img = dodaj_sum(c_img)

    shrani_in_zabelezi(c_img, f"comb_{i}")

print(f" [OK] Iz 30 slik za '{oseba}' je nastalo cca. 450+ različic.")
```

Slika 26 generiranje končnega nabora v predobdelaj_podatke()

Vse posamezne funkcije za transformacijo slike sem združila v glavnem bloku za augmentacijo, ki se izvede za vsako prebrano sliko iz surovega nabora. Zahvaljujoč pomožni funkciji `shrani_in_zabelezi` je koda izjemno pregledna in poteka v naslednjih logičnih sklopih:

- Osnovne različice (Original in Grayscale): Algoritem najprej shrani očiščeno originalno sliko ter njeno sivinsko (Grayscale) različico. Sivinska slika je

pomembna, saj modelom omogoča učenje strukturnih značilnosti obraza (sence, robovi) brez pristranskosti do določenih barvnih tonov ali temperature svetlobe.

- Geometrijske transformacije: Izvede se horizontalno zrcaljenje (cv2.flip), ki simulira zajem s sprednjo "selfie" kamero v načinu ogledala. Sledi zanka, ki sliko zavrti za točno določene kote (-15° , -8° , 8° in 15°), s čimer model postane inavarianten na naravne nagibe glave uporabnika.
- Fotometrične transformacije in degradacija kakovosti: V tem sklopu algoritem generira preosvetljeno (faktor 1.4) in pretemno (faktor 0.6) sliko. Nato uporabi močan Gaussov zameglitveni filter (z velikostjo jedra 7×7), ki simulira neizostreno kamero (out-of-focus) ali premikanje med zajemom, in doda še predhodno opisani zrnati šum.
- Kombinirane naključne transformacije: Ker se v realnosti slabi pogoji pogosto združijo (npr. uporabnik je v temi in hkrati drži telefon postrani), sem implementirala zanko, ki za vsako izhodiščno sliko ustvari še 5 popolnoma unikatnih kombinacij. Z uporabo knjižnice random algoritem naključno izbere kot rotacije, naključno spremeni svetlost in s 50-odstotno verjetnostjo sliki doda digitalni šum.

Rezultat: S tem zaporedjem operacij se vsaka posamezna surova slika transformira v 15 različnih variacij. S tem sem uspešno rešila problem majhnega začetnega nabora podatkov (samo 30 slik na osebo) in za vsakega člana ekipe avtomatsko generirala več kot 450 raznolikih slik, ki so takoj pripravljene za učenje nevronske mreže. V naslednjem poglavju, pa bom razložila še zadnji del funkcije, kjer pripravim učne množice.

5 Priprava učnih množic (Točka 6 navodil)

Skripta samodejno razvrsti vse obdelane podatke v tri ločene množice (mapa final_split):

1. **Učna množica (70 %):** Za dejansko prilagajanje uteži modela.
2. **Validacijska množica (15 %):** Za sprotno spremljanje natančnosti med učenjem.
3. **Testna množica (15 %):** Za končno objektivno oceno uspešnosti sistema na slikah, ki jih model prej ni videl.

5.1 Tehnična izvedba razdelitve podatkov

```
# --- TOČKA 6: PRIPRAVA UČNE, VALIDACIJSKE IN TESTNE MNOŽICE ---
print("\nRazvrščam slike v Train (70%), Val (15%) in Test (15%) množice...")
random.shuffle(vse_generirane_slike)

skupaj = len(vse_generirane_slike)
meja_train = int(skupaj * 0.7)
meja_val = int(skupaj * 0.85)

razdelitev = {
    'train': vse_generirane_slike[:meja_train],
    'val': vse_generirane_slike[meja_train:meja_val],
    'test': vse_generirane_slike[meja_val:]
}

for ime_mnozice, slike in razdelitev.items():
    for pot_vira, oseba in slike:
        pot_cilja = os.path.join(split_path, ime_mnozice, oseba)
        if not os.path.exists(pot_cilja):
            os.makedirs(pot_cilja)
        shutil.copy(pot_vira, os.path.join(pot_cilja, os.path.basename(pot_vira)))

print(f"\nUspeh! Predobdelava končana.")
print(f"Dataset je razdeljen v: {split_path}")
print("Mapi 'obdelani_podatki' in 'final_split' sta ignorirani v Git-u, surove slike pa so pripravljene za Push.")
```

Slika 27 razdelitev podatkov v `predobdelaj_podatke()`

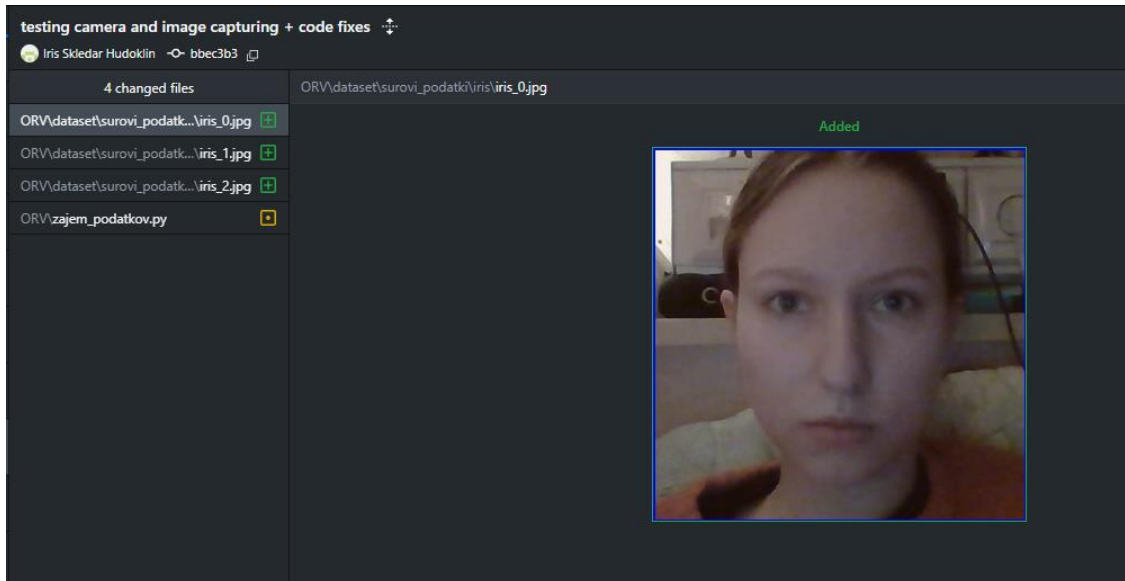
Da bi zadostila strogim zahtevam strojnega učenja, kjer morajo biti podatki za učenje in testiranje popolnoma ločeni, sem implementirala logiko za avtomatsko razvrščanje generiranih slik. Koda deluje po naslednjih korakih:

- Mešanje podatkov (Shuffling): V prejšnjih korakih je skripta vse poti do obdelanih slik pridno shranjevala v globalni seznam `vse_generirane_slike`. Ker so bile slike dodajane zaporedno (najprej vsi originali, nato vse rotacije, itd.), sem seznam na začetku nujno premešala s funkcijo `random.shuffle()`. S tem preprečim pristranskost zaporedja in zagotovim, da so različne augmentacije enakomerno razpršene po vseh množicah.
- Izračun indeksnih mej: S funkcijo `len()` sem pridobila skupno število vseh slik. Nato sem matematično izračunala meje za delitev: `meja_train` predstavlja 70 % celotnega nabora, `meja_val` pa 85 % (70 % za učenje + 15 % za validacijo). Preostanek avtomatsko pripade testni množici (15 %).
- Slovar razdelitve (Dictionary Slicing): Z uporabo Pythonove funkcionalnosti rezanja seznamov (list slicing) sem ustvarila pregleden slovar `razdelitev`. Ta slovar definira, kateri kosi seznama (od katerega do katerega indeksa) pripadajo ključem 'train', 'val' in 'test'.
- Avtomatska gradnja map in kopiranje: V zadnjem koraku se zanka sprehodi čez ustvarjen slovar. Algoritem samodejno preveri, ali ciljna mapa (npr. `dataset/final_split/train/iris`) že obstaja. Če ne, jo zgradi z `os.makedirs()`. Za fizičen premik slik sem uporabila funkcijo `shutil.copy()`, ki varno prekopira sliko iz delovne mape v njeno končno destinacijo.

Na samem koncu skripta v terminal izpiše potrditveno sporočilo in opomnik, da sta mapi obdelani_podatki in final_split vpisani v .gitignore, kar preprečuje pošiljanje tisočev obdelanih slik na repozitorij GitHub, kot je zahtevano v navodilih projekta.

6 Rezultati in dokazi

- Screenshot Android zaslona z ovalnim vodilom je prikazan v poglavju 2.1 Android Vmesnik.
- Screenshot commita prvega testa skripte zajem_podatkov.py



Slika 28 commit testiranja zajem_podatkov.py

- Primerjalna slika: Surova slika vs. obdelana zrnata/rotirana slika.

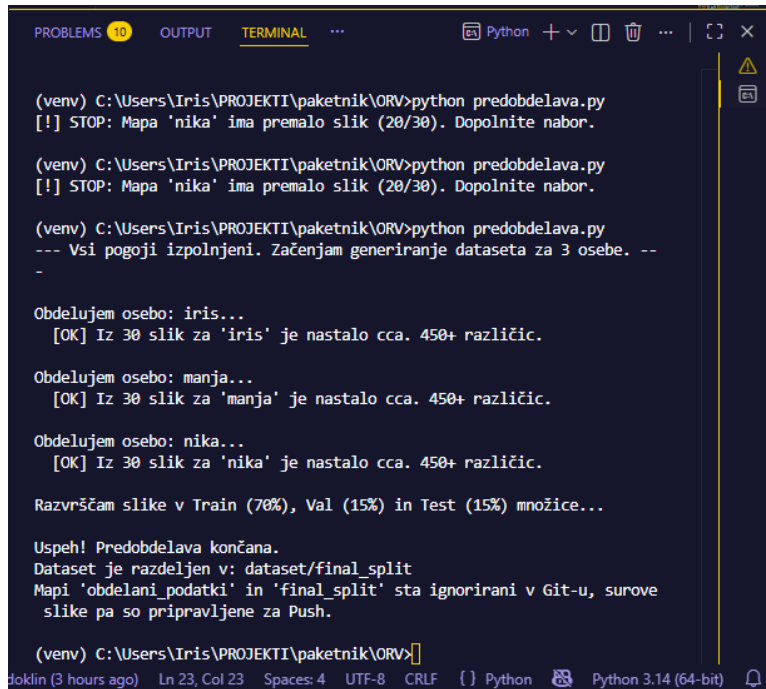


Slika 29 surova slika



Slika 30 obdelana slika

- Screenshot terminala po neuspešnem in uspešnem zagonu predobdelava.py (kjer piše "Vsi pogoji izpolnjeni").



```
(venv) C:\Users\Iris\PROJEKTI\paketnik\ORV>python predobdelava.py
[!] STOP: Mapa 'nika' ima premalo slik (20/30). Dopolnite nabor.

(venv) C:\Users\Iris\PROJEKTI\paketnik\ORV>python predobdelava.py
[!] STOP: Mapa 'nika' ima premalo slik (20/30). Dopolnite nabor.

(venv) C:\Users\Iris\PROJEKTI\paketnik\ORV>python predobdelava.py
--- Vsi pogoji izpolnjeni. Začenjam generiranje dataseta za 3 osebe. --
-

Obdelujem osebo: iris...
[OK] Iz 30 slik za 'iris' je nastalo cca. 450+ različic.

Obdelujem osebo: manja...
[OK] Iz 30 slik za 'manja' je nastalo cca. 450+ različic.

Obdelujem osebo: nika...
[OK] Iz 30 slik za 'nika' je nastalo cca. 450+ različic.

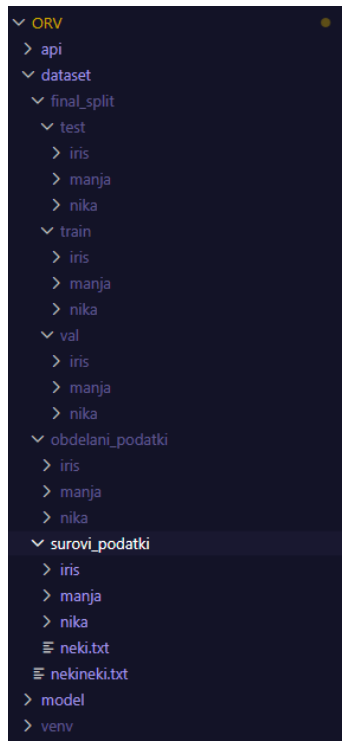
Razvrščam slike v Train (70%), Val (15%) in Test (15%) množice...

Uspeh! Predobdelava končana.
Dataset je razdeljen v: dataset/final_split
Mapi 'obdelani_podatki' in 'final_split' sta ignorirani v Git-u, surove
slike pa so pripravljene za Push.

(venv) C:\Users\Iris\PROJEKTI\paketnik\ORV>
```

Slika 31 terminal ob zagonu predobdelava.py

- Screenshot mape final_split, ki prikazuje strukturo train/val/test.



Slika 32 struktura map

- Screenshot priprave zaslona FaceIdActivity kot vhodno točko slik

```
23
24 class FaceIdActivity : AppCompatActivity() {
25
26     private lateinit var binding: ActivityFaceIdBinding
27     private lateinit var cameraExecutor: ExecutorService
28
29
30     override fun onCreate(savedInstanceState: Bundle?) {
31         super.onCreate(savedInstanceState)
32
33         binding = ActivityFaceIdBinding.inflate(layoutInflater)
34         // Edge-to-Edge
35         enableEdgeToEdge()
36         setContentView(binding.root)
37         cameraExecutor = Executors.newSingleThreadExecutor()
38
39
40         //odmiki zaradi statusne vrstice in prekrivanja napisa
41         ViewCompat.setOnApplyWindowInsetsListener(binding.root) { v, insets ->
42             val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
43             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
44             insets
45         }
46
47         startCamera()
48
49         //Gumb za nazaj
50         binding.btnBack.setOnClickListener {
51             finish()
52         }
53         binding.btnHome.setOnClickListener {
54             val intent = Intent(this, MainActivity::class.java)
55             intent.flags = Intent.FLAG_ACTIVITY_CLEAR_TOP or Intent.FLAG_ACTIVITY_SINGLE_TOP
56             startActivity(intent)
57             finish()
58         }
59         // Povezava gumb capture
60         binding.btnCapture.setOnClickListener {
61             // Tukaj bo kasneje logika za zajem slike za ORV
62             Toast.makeText(this, "Obraz zajet!", Toast.LENGTH_SHORT).show()
63         }
64     }
65
66     private fun startCamera() {
67         val cameraProviderFuture = ProcessCameraProvider.getInstance(this)
68
69         cameraProviderFuture.addListener({
70             val cameraProvider: ProcessCameraProvider = cameraProviderFuture.get()
71
72             val preview = Preview.Builder()
73                 .build()
74                 .also {
75                     it.setSurfaceProvider(binding.viewFinder.surfaceProvider)
76                 }
77
78             //spredna kamera
79             val cameraSelector = CameraSelector.DEFAULT_FRONT_CAMERA
80
81             try {
82                 cameraProvider.unbindAll()
83                 cameraProvider.bindToLifecycle(this, cameraSelector, preview)
84
85             } catch (exc: Exception) {
86                 Log.e("FaceID", "Zagon kamere ni uspel", exc)
87             }
88
89             }, ContextCompat.getMainExecutor(this))
90     }
91
92     override fun onDestroy() {
93         super.onDestroy()
94         cameraExecutor.shutdown()
95     }
96 }
```

Slika 33 FaceIdActivity

- Screenshot navodil v ORV/README.md za ostale člane

6d81590

paketnik / ORV / README.md

Go to file

IrisSkledar

Updated the README file and added final_split to gitignore

6d81590 · last week

59 lines (39 loc) · 2.15 KB

PreviewCodeBlame

Raw

ORV - Računalniški vid (Zajem in obdelava)

Ta mapa vsebuje Python orodja za vzpostavitev nabora učnih podatkov (dataset) za model razpoznave obrazov.

1. Priprava okolja

V terminalu se postavite v mapo `ORV` in izvedite naslednje korake:

1. Ustvari virtualno okolje: `python -m venv venv`
2. Aktiviraj okolje glede na tvoj sistem:

Sistem / Terminal	Ukaz
Windows PowerShell	<code>.\venv\Scripts\Activate.ps1</code>
Windows CMD	<code>venv\Scripts\activate</code>
macOS / Linux	<code>source venv/bin/activate</code>

Ko je okolje aktivirano, boste v terminalu videli oznako `(venv)`.

3. Namesti knjižnice: `pip install -r requirements.txt`

2. Postopek zajema podatkov (Vsaka članica)

Vsaka članica ekipe mora v bazo prispevati **točno 30** slik svojega obraza.

1. Zaženi: `python zajem_podatkov.py`
2. Vpiši svoje ime (npr. `manja` ali `nika`).
3. Postavi se pred kamero (zelen okvir mora zaznati obraz).
4. Pritisni 's' za zajem posamezne slike (ponavljaj do 30).
5. Pritisni 'q' za izhod.

Pomembno: Surove slike iz mape `dataset/surovi_podatki/` commitajte in pushajte na GitHub, da jih dobijo ostale članice.

3. Predobdelava in generiranje učnih podatkov (Član 2)

Ko so v mapi `surovi_podatki` zbrani zajemi vseh treh članic (vsaj 3 mape, vsaka s 30 slikami), Član 2 zažene postopek:

1. Zaženi: `python predobdelava.py`
2. Kaj skripta naredi:
 - Odstranjevanje šuma: Slike očisti z Gaussovimi filtrom (denoising).
 - Augmentacija: Iz 90 slik zgenerira ~1300+ različic (rotacije, svetlost, šum, zameglitev).
 - Split: Podatke samodejno razdeli v razmerju **70% Train / 15% Val / 15% Test**.
3. Rezultat: Končni podatki za učenje modela se nahajajo v mapi `dataset/final_split/`.

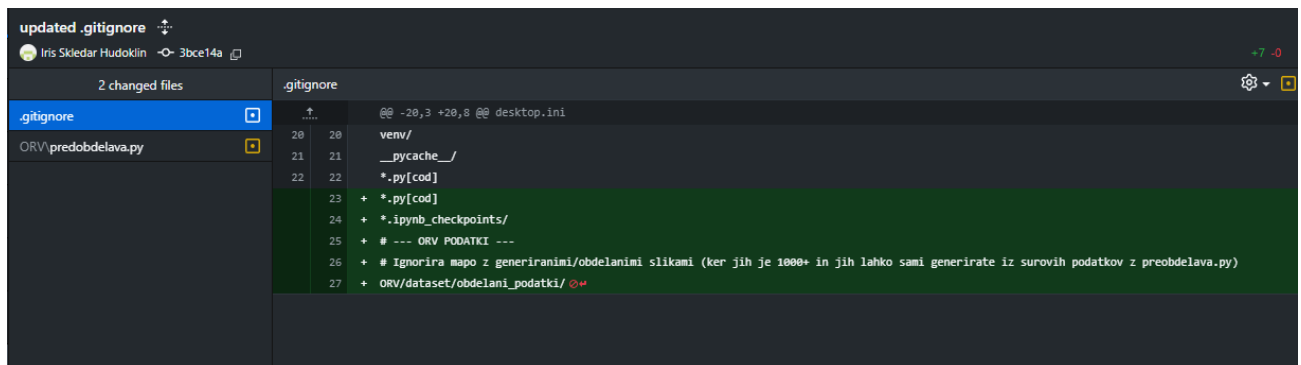
Opomba: Mapi `obdelani_podatki` in `final_split` sta v `.gitignore` in se ne pošiljata na GitHub zaradi velikosti.

Zahteve za delovanje skript

- Število oseb: Vsaj 3 (iris, manja, nika).
- Število slik: Vsaka oseba mora imeti točno 30 surovih slik.
- Validacija: Skripta `predobdelava.py` bo prekinila postopek, če pogoji niso izpolnjeni.

Slika 34 README.md navodila

- Screenshot dodajanja mape obdelani_podatki v .gitignore



```

updated .gitignore
Iris Skledar Hudoklin 3bce14a

2 changed files
.gitignore
ORV/preobdelava.py

.gitignore
↑ @@ -20,3 +20,8 @@ desktop.ini
20 venv/
21 __pycache__/
22 *.py[cod]
23 + *.py[co]
24 + *.ipynb_checkpoints/
25 + # --- ORV PODATKI ---
26 + # Ignorira mapo z generiranimi/obdelanimi slikami (ker jih je 1000+ in jih lahko sami generirate iz surovih podatkov z preobdelava.py)
27 + ORV/dataset/obdelani_podatki/
  
```

7 Zaključek

V okviru projektne naloge sem kot Član 1 uspešno zasnovala in implementirala sistem za zajem, predobdelavo in augmentacijo slikovnih podatkov. Začetni izziv, strah pred premajhim številom fizično zajetih slik (30 na osebo) sem rešila z razvojem robustnega avtomatiziranega cevovoda, ki je z uporabo geometrijskih in fotometričnih transformacij generiral uravnotežen nabor več kot 1300 variacij.

S strogim preverjanjem pogojev (validacijo) ob zagonu skripte in avtomatsko razdelitvijo podatkov v učne, validacijske in testne množice sem zagotovila varno izhodišče za Člana 2 pri učenju modela strojnega učenja. Prav tako sem z integracijsko funkcijo (pripravi_slike_za_api) poskrbel, da bo končna aplikacija (naloge Člana 3) prejete slike iz mobilne kamere obdelala na format ki ga model pričakuje (obrezovanje, resize, čiščenje), kar je ključno za visoko natančnost sistema v produkciji.

Cilji, določeni v navodilih za Člana 1, so bili doseženi, razvita koda pa je optimizirana, zaščitena pred napakami in pripravljena na integracijo v končno rešitev pametnega paketa.

8 Viri

- OpenCV. Cascade Classifier. Dostopno na:
https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- OpenCV (2024). Image Processing in OpenCV. Dostopno na:
https://docs.opencv.org/4.x/d7/d1b/group_imgproc_misc.html
- Rosebrock, A. Face Detection with OpenCV and Haar Cascades. PyImageSearch. Dostopno na:<https://pyimagesearch.com/2021/04/05/opencv-face-detection-with-haar-cascades/>
- OpenCV. Geometric Image Transformations. Uradna dokumentacija za funkciji getRotationMatrix2D in warpAffine. Dostopno na:
https://docs.opencv.org/4.13.0/da/d54/group_imgproc_transform.html
- NumPy. NumPy Reference. Uradna dokumentacija za numerično obdelavo, upravljanje s slikovnimi matrikami in generiranje Gaussovega šuma (numpy.random). Dostopno na: <https://numpy.org/doc/stable/reference/>
- Python Software Foundation. The Python Standard Library. Uradna dokumentacija za vgrajene knjižnice za upravljanje z datotečnim sistemom in mapami (os, shutil) ter naključnimi operacijami (random). Dostopno na:
<https://docs.python.org/3/library/>